Project Proposal: A Compiler from StragglAR Schedules to NCCL Send/Receive Programs

Team 31 (Chesterjesters): Wyatt Sell, David Valarezo, Nathnael Tesfaw, Aneesh Naresh
Advisor: Arjun Devraj · Compute: 2 Perlmutter nodes (8×A100, NCCL over Slingshot)

Goal. Build a compiler that reads StragglAR's synthesized schedule (rounds of *StragglerMatching*, *OneWayMatching*, and *TwoWayMatching* transfers) and generates a C++/CUDA program of grouped `ncclSend`/`ncclRecv` calls with local reductions, one process per GPU. We aim to achieve a program that runs without deadlock on 8 GPUs across two nodes, gives the same output as `ncclAllReduce`, and on 4 GPUs (one node) finishes faster than `ncclAllReduce` when the same straggler delay is injected into both. Beating it on 8 GPUs across two nodes is a stretch goal, since StragglAR assumes equal-speed links between all GPUs.

Pipeline. StragglAR schedule → parser → IR → code generator → NCCL program → test harness on Perlmutter.

Member	Owns	Poster panel
Wyatt Sell	Generating schedules for $n = 4, 8, 16$; schedule parser into an IR (per round: sender, receiver, chunk, add or copy) and a spec of both; ReduceScatter phase before the schedule (communicator without the straggler); straggler remap so whichever GPU is slow takes rank $n-1$; CPU simulator that checks the schedule and generated code (matched sends/receives, correct sums).	Problem, StragglAR background, correctness checking.
David Valarezo	Code generator: IR → C++/CUDA with one `ncclGroupStart/End` block per round; mapping chunks to buffer positions and scratch buffers; per-GPU code, one process each; deadlock-free ordering; reusing the repo's reduction kernel; checked against the repo's hand-written code.	Compiler design and a generated-code example.
Nathnael Tesfaw	Owns the launch scripts (Slurm, 1 and 2 nodes); NCCL build and Makefile; multi-process NCCL setup (repo code is single-process); a hand-written send/recv test on 8 GPUs.	Experimental setup and system details.
Aneesh Naresh	Test harness, adapted from the repo's timing code: buffer init, comparison against `ncclAllReduce`, timing ours vs. `ncclAllReduce` with the same delay on rank $n-1$, and with the delay on a different GPU to test the straggler remap; result plots.	Results: correctness and speedup vs. `ncclAllReduce` under a straggler.

Timeline.

Dates	Work
Oct 26 – Oct 29	All: kickoff, read the synthesizer and hand-written programs. Wyatt generates schedules and drafts the schedule-format spec. Nathnael starts the NCCL build and launch script.
Oct 30 – Nov 8	In parallel: Wyatt delivers the parser and IR by Oct 31, then builds the simulator and ReduceScatter phase; David generates code from the n=4 IR; Nathnael delivers the launch script (Nov 1) and the 8-GPU send/recv test; Aneesh builds the harness against `ncclAllReduce`.
Nov 9	**Midpoint presentation**: schedules for several n, the format and IR spec, generated code for n=4 running on one node and passing the harness. Each member presents their stage.
Nov 10 – Nov 22	Full pipeline for n=8 on two nodes. Wyatt checks generated code in the simulator before GPU runs and adds the straggler remap; Nathnael and Aneesh run and debug on 8 GPUs. Goal: no deadlock and correct results by Nov 22.
Nov 23 – Dec 1	Aneesh and Nathnael time the generated program against `ncclAllReduce` on 4 and 8 GPUs with the same straggler delay on both (sweeping delay × buffer size). Wyatt and David tune the generated code if we are not faster, otherwise finish the docs. Thanksgiving week is slack.
Dec 2 – Dec 6	Each member makes their poster panel; Dec 5 full team review and practice.
Dec 7	**Final poster session.**

Deliverables. Schedules for several power-of-two n; the compiler and a spec of the schedule format and IR; a test showing the generated program runs without deadlock on 8 GPUs across two nodes, matches `ncclAllReduce`, and beats it on 4 GPUs under the same straggler delay (8 GPUs as a stretch goal).

AI disclosure. This proposal was developed with Claude (Anthropic). Claude produced a first draft from the project brief and was used to understand the StragglAR code; we then revised it over many rounds, setting the success criteria, deciding the compiler's structure, rebalancing the division of work, and cutting and reformatting the content.